accurately reflect the unpredictable nature of real model outputs. After all, the developer crafts the actual responses instead of the generative AI model. There is an option to set up a small LLM for local testing, but in that case our tests would be considered more like integration tests instead of unit tests, so our expectations would change.

It is also essential to remember that mocking code under your control can obscure problems within your own implementation. What you should do instead is focus on mocking external dependencies or components that are not the primary focus of the test. Be aware of these considerations when writing unit tests so that the quality of your test cases won't degrade as you develop your applications further.

8.4.4 Testing LangChain.js

When working with LangChain.js, testing the responses from language models and the behavior of chains helps ensure your application functions as intended. There are many considerations, but they basically boil down to the type of code we have introduced as part of using LangChain.js. To test relevant functionality effectively, use mocking to simulate the behavior of AI models and ensure that your functions behave as expected under various scenarios. Here's how we can approach testing each part of the code:

- *Mocking dependencies*—Use mocking to simulate the behavior of external libraries and APIs, such as `ChatGoogleGenerativeAI`, `PDFLoader`, and `DocxLoader`. This allows us to isolate the logic of our application from external dependencies since it's not our responsibility to cover unit tests of external libraries.
- *Unit testing functions*—Each function can be tested individually to ensure it performs its tasks correctly. For instance, we can test that functions like `normalizeDocuments` correctly format document content or that `processFile` correctly handles different file types.
- *Integration testing*—For functions that involve multiple components or chains, such as `summarizeDocs`, integration tests can verify that these components work together as expected.

Let's test a few functions and helpers we used in the previous chapters that included LangChain.js.

TESTING HELPERS FROM LANGCHAIN

LangChain provides several testing helpers, summarized in table 8.2, to simulate the behavior of language models, loaders, retrievers, and other components. This allows developers to create test cases without relying on actual API calls or external dependencies.

Let's see how to create unit tests for the `performRAG` method in the `RAGSystem` class that we created in chapter 7, utilizing the `FakeLLM` and `FakeRetriever` classes from LangChain (see listing 8.9). We will ensure that the `performRAG` method behaves correctly under various scenarios by simulating the behavior of the language model and retriever. To do that, we replace the inner properties of this class with the respective `FakeLLM` and `FakeRetriever` instances.